Mean-Safe Is Not Tail-Safe: Token-Wise Latent Capacity in Multi-Head Latent Attention

ShangJae Lee
Kyungpook National University
dltkdwo31@knu.ac.kr

Abstract

Multi-head Latent Attention (MLA) caches one compressed latent per token, at the same width for every token. How much of that width does each token need? We cast token-wise latent width as a rate allocation problem with two exact properties: cache bytes are affine in the summed per-token rates, so allocations with equal rate sums cost exactly the same, and requiring a token to be safe on the worst k of its future losses, a CVaR-type criterion, orders the safe rates monotonically from the average to the worst reuse offset. Measured on three MLA language models (30M-250M parameters) and on the public, production-trained DeepSeek-V2-Lite, this risk-capacity spectrum is wide and stable. Under a 0.1-nat budget the safe rate averages 7-9% of the latent width when only the mean future loss is constrained, but 72-79% when the worst future offset is, and the normalized gap between the two is nearly constant (0.646-0.651 across our three models, 0.70 in DeepSeek-V2-Lite). The gap is not caused by rare catastrophic tokens: 93-97% of mean-safe tokens still exceed the budget at some future offset, and their mean stays small because losses and gains cancel across the horizon. Rotating the latent basis lowers the worst-offset requirement by about 50 points in our small models but by only 11 in DeepSeek-V2-Lite. We then ask whether a learned router can exploit the spectrum, using a variable-width packed cache and pre-registered exact-byte controls. At equal bytes the router beats random, position-based and token-type placement in three of four configurations, but only ties a per-token lookup table: what it learned is explained by token identity, not by context.

1 Introduction

Autoregressive transformer [1] inference stores a key and a value for every past token in every attention layer, and this KV cache, growing linearly with context length and batch size, increasingly limits serving capacity. Multi-head Latent Attention (MLA), introduced in DeepSeek-V2 [2] and retained in DeepSeek-V3 [3], shrinks the cache by storing one low-dimensional latent per token from which per-head keys and values are reconstructed. MLA nevertheless gives every token the same latent width. A natural next step, analogous to adaptive computation [26, 27] and to per-token budget allocation in cache eviction [18, 20], is to give each token only the width it needs. Before building such a system, one has to know what "needed" means and how much it varies.

That depends on the criterion. A cached latent is read by every later query, so truncating it changes the loss at many future positions at once. A token can be "safe" in the sense that its average effect over the next H positions is small, or in the stricter sense that no single future position is harmed by more than a budget ϵ. The usual intuition is that the two nearly coincide: most tokens are harmless, and a few rare tokens cause large spikes. If that were true, a router could compress aggressively and protect the rare spikes.

We test this directly and find that it is not true. Our contributions are:

- **A rate-allocation formulation with two exact properties** (Section 3). The persistent bytes of a variable-width packed MLA cache are an affine function of the summed per-token rates, so any two

allocations with the same rate sum cost exactly the same bytes; this is what makes byte-exact comparisons between allocation policies possible. And the safe rate under a "worst k of H future losses" criterion, the empirical conditional value-at-risk [33] of a token's future-loss vector, is monotone in k for every token and every realized loss curve, without any smoothness or monotonicity assumption.

- **The risk-capacity spectrum** (Section 5). On three MLA models spanning an 8x parameter range and on DeepSeek-V2-Lite, the mean safe rate rises from 7-9% of the latent width under the mean criterion to 72-79% under the worst-offset criterion, through every intermediate tail level, and the normalized tail-capacity premium is nearly scale-invariant (0.646-0.651; 0.704 for DeepSeek-V2-Lite).
- **A mechanism.** The gap comes from pervasive cancellation, not rare spikes: at the rate that is safe on average, 93-97% of tokens still exceed the budget at some future offset, and the positive-part mean loss is 2.05-2.31x the signed mean.
- **A basis-dependence result with a negative half.** In our models a learned rotation of the latent basis removes about 50 points of the worst-offset requirement, but in DeepSeek-V2-Lite only 11: the tail requirement is real, while the large rotation gain is specific to small models trained on TinyStories.
- **A pre-registered case study of learned allocation** (Section 6). A contextual router with a variable-width packed cache beats exact-byte random allocation in three of four frozen configurations and beats position and token-type placement at exactly the router's bytes, but it only ties a per-token lookup table fitted to its own decisions.

The spectrum is the paper's main result and does not depend on the router. The router result is narrower, and we report it as it is: a learned allocator can beat naive placements, but we find no evidence that context beyond the current token helps it. We also do not claim serving gains. In our prototype the packed cache stores 60-67% of the bytes of a full MLA cache, with essentially unchanged peak memory and a 2.4x decode-latency cost (Section 7).

2 Background and Related Work

2.1 Multi-head Latent Attention

For hidden state h_t, MLA computes a compressed KV latent

$$c_t = W_{DKV} h_t \in \mathbb{R}^{d_c},$$

and reconstructs content keys and values as $W_{UK} c_t$ and $W_{UV} c_t$. A separate rotary key $k_t^R \in \mathbb{R}^{d_R}$ carries positional information. A fixed-width MLA cache stores (c_t, k_t^R) instead of per-head keys and values; for sequence length T, L layers and element size b its leading storage term is $bTL(d_c+d_R)$. MLA continues a line of work that shares or compresses key-value heads, including multi-query [22] and grouped-query attention [23]. MHA2MLA [4], TransMLA [6] and CARE [8] convert pretrained multi-head or grouped-query models to MLA, the last allocating rank across layers with a covariance-aware decomposition; EG-MLA [7] gates latent content with a token-embedding signal; and Cross-Layer Latent Attention combines latent compression with layer sharing and quantization [5]. Analyses of trained MLA models find that latent coordinates separate content from positional information in a way that depends on the basis used to read them [9]. All of these keep one latent width per layer for every token.

2.2 KV-cache compression

KV-cache compression methods differ in which axis they compress. Eviction keeps a subset of tokens, selected by accumulated attention (H2O [10], Scissorhands [19]), attention sinks and a recent window (StreamingLLM [11]), or attention patterns observed during prefill (SnapKV [12]); FastGen [20] picks a policy per head, and PyramidKV [13] and Ada-KV [18] allocate different budgets to different layers or heads. Quantization lowers the precision of every cached element (KIVI [14], KVQuant [15]). Low-rank methods project keys and values into a smaller space, uniformly or with layer-wise ranks (Palu [16], LoRC [17], Eigen Attention [21]), and MiniCache [24] merges states across adjacent layers. Dynamic Memory Compression [25] learns per token whether to append or merge into the cache.

Our object is different from each of these. We keep every token and every head, and vary the *width* of each token's latent through a nested prefix, the same nesting idea as Matryoshka representations [30]. This makes per-token rate the only thing that varies, which is what lets us measure how much each token needs and compare allocation policies at byte-identical budgets.

2.3 Adaptive allocation and its evaluation

Learning where to spend a discrete budget is a general pattern: adaptive computation time [26], mixture-of-depths [27] and mixture-of-experts routing all learn an importance signal and spend compute nonuniformly. Our router is trained with a straight-through estimator [28] through the hard tiers it uses at deployment. Two evaluation points from this setting carry over. First, an allocation policy should be compared with alternatives at exactly its own budget, or the comparison measures budget rather than placement. Second, the alternatives should include simple causal rules, not only random placement: a router can beat random placement while learning nothing that a lookup table could not. Section 6 applies both.

3 Token-Wise Latent Rate Allocation

3.1 Nested latent prefixes and packed storage

For each layer ℓ, calibration produces a permutation π_ℓ of the d_c latent channels by gradient-times-activation saliency. A token assigned rate r_t stores the nested prefix

$$\tilde{c}_{\ell,t} = c_{\ell,t}[\pi_\ell(1 : r_t)].$$

The packed cache stores a flat values buffer, int32 offsets, int16 ranks, an int16 channel permutation and the uncompressed RoPE key. Layer 0 is always kept at full width (Section 6.1). For a batch-one sequence its exact tensor-payload byte count is

$$M_{\text{packed}} = b\,[Td_c + (L-1)T\bar{r} + TLd_R] + 4L(T+1) + 2LT + 2Ld_c,$$

where $\bar{r} = T^{-1}\sum_t r_t$ and the last three terms count offsets, ranks and permutations; we use $b = 4$. This counts the persistent tensors only, not allocator rounding, router weights or the dense temporary latent that the prototype reconstructs during attention (Section 7 measures those).

3.2 Two propositions

We do not call r_t an intrinsic rank. MLA's latent can be rotated while compensating the up-projections ($c \mapsto Qc, W_U \mapsto W_U Q^{-1}$), and a nested coordinate prefix is not invariant under this change: the full model is, its truncations are not. r_t is therefore an operational rate in a chosen nested codebook $P_{\ell,r_1} \preceq \cdots \preceq P_{\ell,d_c} = I$, and Section 5.4 measures how much the choice of codebook matters.

Table 1: The three MLA language models trained for this study, all on TinyStories.

Model	Parameters	Layers	d_{model}	Heads	d_c	d_R	Context	Training tokens
MLA-30M	30.6M	6	384	6	256	32	256	24.6M
MLA-122M	122.14M	12	768	12	384	32	384	147.5M
MLA-250M	249.27M	16	1024	16	512	32	384	55.3M

Proposition 1 (exact budget affinity). For any two per-sequence allocations $\mathbf{r}, \mathbf{s}$, $M_{\text{packed}}(\mathbf{r}) - M_{\text{packed}}(\mathbf{s}) = b(L-1)\sum_t (r_t - s_t)$. In particular, allocations with the same rate sum have identical cache bytes. *Proof.* Immediate from the byte formula, whose only allocation-dependent term is $b(L-1)T\bar{r}$.

Proposition 1 is what licenses exact-byte controls: an alternative placement of the same rate sum is a comparison at precisely the same memory, not an approximately matched one.

Let $\Delta_{t,j}(r) = \ell_{t+j}(P_r c_t) - \ell_{t+j}(c_t)$ be the signed change in next-token cross-entropy at future offset $j \in \{0, \ldots, H-1\}$ when only source token t is truncated to rate r. For a tail count k, let $A_{t,k}(r)$ be the mean of the k largest of the H values $\Delta_{t,j}(r)$. This is the empirical conditional value-at-risk [33] of the future-loss vector at level $\alpha = 1 - k/H$: $k = H$ gives the signed mean, $k = 1$ the maximum. Define the suffix-safe rate

$$r^*_{t,k}(\epsilon) = \min\{r \in \mathcal{R} : A_{t,k}(r') \le \epsilon \text{ for every } r' \in \mathcal{R},\ r' \ge r\},$$

the smallest rate from which every larger rate on the grid $\mathcal{R}$ is also safe.

Proposition 2 (risk-capacity ordering). If $k_1 \ge k_2$, then $r^*_{t,k_1}(\epsilon) \le r^*_{t,k_2}(\epsilon)$ for every token, budget and grid. *Proof.* The mean of the k largest elements of a fixed vector cannot decrease as k shrinks, so $A_{t,k_1}(r) \le A_{t,k_2}(r)$ at every r. Every rate that is suffix-safe for k_2 is therefore suffix-safe for k_1, and the minima are ordered.

The ordering holds for every realized loss vector, including the nonmonotone rank-loss curves we observe (Appendix G), and needs no assumption about the sign or shape of $\Delta_{t,j}$. It is why the result in Section 5 is a whole spectrum of tail levels rather than two endpoints. Under a bounded-Lipschitz assumption on the future logit map, truncation also satisfies $|\Delta_{t,j}(r)| \le \sqrt{2} K_{t,j} \|e_t(r)\|_2$, where $e_t(r)$ is the discarded latent tail and $K_{t,j}$ a local Lipschitz constant: a monotone safety bound consistent with, but weaker than, the realized ordering.

3.3 Measuring safe rates

For each probed source position p and each rate r on the grid, we zero the channels outside the layer's nested prefix at position p in every layer simultaneously, keep every other position at full width, and record $\Delta_{p,j}(r)$ for $j = 0, \ldots, H-1$, starting with the prediction made at position p itself, the first one the truncated latent can affect. We use $H = 32$ and $\epsilon = 0.10$ nat. Calibration data, calibration windows and evaluation windows do not overlap. The rate grid runs from 16 to d_c in steps of 16 up to 64 and steps of 32 thereafter, so every requirement is resolved up to full width. We report, for each tail count $k \in \{32, 16, 8, 4, 2, 1\}$, the mean of $r^*_{p,k}$ over probes divided by d_c, and the tail-capacity premium $\text{TCP} = \mathbb{E}[r^*_{p,1} - r^*_{p,32}]/d_c$, the average extra width a token needs to be safe at its worst offset rather than on average.

4 Experimental Setup

4.1 Models and data

We train three decoder-only MLA language models on TinyStories [29], spanning an 8x parameter range.

Table 2: Mean suffix-safe rate as a fraction of d_c at each tail level ($\epsilon = 0.10$ nat, $H = 32$), in the saliency-ordered coordinate basis. TCP is the tail-capacity premium.

k (of 32)	$\alpha = 1 - k/H$	30M	122M	250M	DeepSeek-V2-Lite
32 (mean)	0.000	9.00%	8.22%	7.15%	8.22%
16	0.500	21.47%	23.00%	22.12%	22.92%
8	0.750	39.65%	39.52%	38.60%	40.14%
4	0.875	55.75%	54.22%	54.16%	56.86%
2	0.938	66.94%	65.56%	64.95%	70.69%
1 (max)	0.969	74.07%	72.92%	71.78%	78.65%
TCP		0.651	0.647	0.646	0.704

Final validation losses are 1.9618 (30M), 1.5662 (122M) and 1.6588 (250M); every model is its final training step. The 250M model saw fewer tokens than the 122M model and has a higher loss, so every cross-scale comparison in this paper confounds size with training budget. Each scale is probed on 24 held-out sequences with 32 positions each (768 probes).

To test whether the findings survive outside our own training runs, we apply the identical probe to the public DeepSeek-V2-Lite checkpoint [2]. It caches exactly the object studied here: a 512-wide latent, the same d_c as our 250M model, plus a 64-wide RoPE key, in a 27-layer model with 15.7B total and 2.4B active parameters trained by its authors on 5.7T tokens. We train nothing on it. Because TinyStories is out of distribution for this model, its text is WikiText-103 [31]: 24 test sequences of 512 tokens after a BOS token that is never probed, 16 positions each (384 probes), with 32 training sequences for calibration. The latent is truncated through a forward hook on the normalization whose output the cache stores, at every layer at once; the RoPE key is left intact, as in our models. The model runs in float16 on two T4 GPUs; every basis reproduces the unmodified loss exactly at full width (maximum deviation 0.0), and the unmodified mean token loss is 2.14 nats.

4.2 Statistical protocol

The unit of inference is a sequence. Intervals are 95% sequence-cluster bootstrap intervals, and comparisons between methods measured on the same probes or sequences use paired differences. For the router (Section 6), each configuration's policy, sampling seed, 24 fresh evaluation sequences, control repeats, primary endpoint and success rule were frozen in a committed protocol before its confirmation run, and an independent audit script recomputes every reported statistic from the stored per-sequence rows. Appendix B gives the protocol and its two documented departures from a clean pre-registration.

5 Mean-Safe Is Not Tail-Safe: The Risk-Capacity Spectrum

5.1 The spectrum is wide, monotone and nearly scale-invariant

Table 2 gives the mean suffix-safe rate at each tail level, in the saliency-ordered coordinate basis that the packed cache uses. A token whose average future-loss increase must stay under 0.1 nat needs on average 7-9% of the latent; the same token, required to stay under 0.1 nat at every one of its next 32 positions, needs 72-79%. Every intermediate level lies between them, and every step is monotone at every scale, as Proposition 2 guarantees.

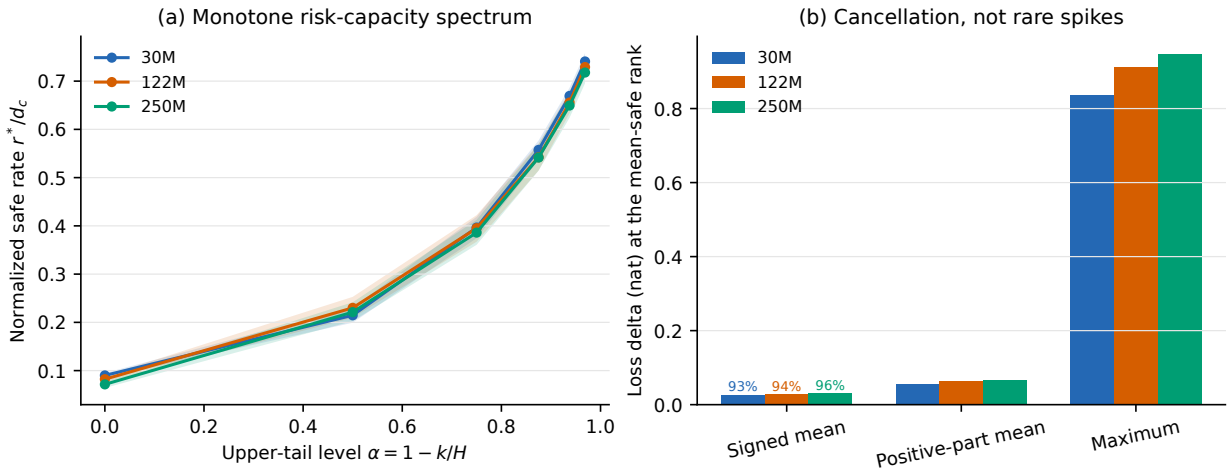


Figure 1: (a) Normalized suffix-safe rate against upper-tail level $\alpha = 1 - k/H$ for the three trained models, with paired sequence-bootstrap 95% bands; monotonicity is guaranteed by Proposition 2. (b) Signed mean, positive-part mean and maximum loss change at the rate that is safe under the signed-mean criterion, annotated with the fraction of probes having at least one future offset above ϵ; the gap between the signed and positive-part means is cancellation, not rare spikes.

The normalized gap is the most scale-consistent quantity we measured. The tail-capacity premium is 0.6507 (95% CI [0.6326, 0.6669]) at 30M, 0.6470 ([0.6283, 0.6652]) at 122M and 0.6464 ([0.6257, 0.6665]) at 250M: three overlapping intervals across an 8x parameter range and two different training budgets. Figure 1(a) shows the three spectra with their bands.

5.2 The gap comes from cancellation, not rare spikes

If the gap were caused by a few catastrophic tokens, most tokens that are safe on average would also be safe at every offset. They are not. At the rate that is safe under the mean criterion, 93.1% (30M), 94.1% (122M) and 96.1% (250M) of probes still have at least one future offset whose loss increase exceeds ϵ, and the positive-part mean loss, the mean of $\max(\Delta, 0)$ over the horizon, is 2.05x, 2.31x and 2.15x the signed mean (Figure 1(b)). Truncating a token's latent hurts some future predictions and helps others; the signed mean is small because the two cancel, not because the harm is small. A mean-criterion router therefore does not under-provide for a few rare tokens. It under-provides for almost every token, at some offset.

5.3 The spectrum replicates on DeepSeek-V2-Lite

On DeepSeek-V2-Lite (last column of Table 2) the spectrum is monotone and close to ours at every tail level, somewhat heavier at the far tail: its premium, 0.704 (95% CI [0.676, 0.731]), is above the 0.646-0.651 of our three models and the intervals do not overlap, so we claim the same shape and magnitude, not the same value. The cancellation signature also replicates: at the mean-safe rate the positive-part mean loss is 2.10x the signed mean, and 96.6% of probes still have an offset above ϵ. One qualification: 275 of the 384 coordinate-basis probes reach the mean criterion at the grid's smallest rate (16, 3.1% of d_c), so the 8.22% mean is an upper bound and the premium a lower bound.

Table 3: Worst-offset safe rate in the coordinate basis and after rotating the latent to principal directions, with the paired per-probe fall (95% CI shown for DeepSeek-V2-Lite).

Model	Coordinate basis, max	Principal directions, max	Paired fall (points)
30M	74.07%	23.56%	50.5
122M	72.92%	23.25%	49.7
250M	71.78%	21.89%	49.9
DeepSeek-V2-Lite	78.65%	67.34%	11.3 [9.1, 13.5]

5.4 How much of the tail requirement is the basis?

Every rate above is measured in the model's own latent coordinates, ordered by saliency: a permutation, the most restricted nested codebook. We reran the identical probe in rotated orthonormal bases calibrated on the same tokens: principal directions of $\mathbb{E}[cc^\top]$ ("pca"), eigenvectors of the first-order loss-effect form $\mathrm{sym}(\mathbb{E}[gc^\top])$ ("grad"), and principal directions in the metric the up-projections apply to the latent ("model metric"). The rotation is folded into the weights, so it leaves the full-width model unchanged and makes a prefix mask a projection onto the leading directions (Appendix D).

In our three models (Table 3) rotation removes about 50 points of the worst-offset requirement at every scale, with paired intervals far from zero; at 250M the coordinate basis needs the entire latent for 77 of 768 probes and principal directions for none. In DeepSeek-V2-Lite it removes 11.3 points (principal directions; 11.1 for the model-metric frame, which ties principal directions with a paired interval of [-1.3, +1.9]), and 59 of 384 probes still need the entire latent (106 in coordinates). Rotation helps at every tail level in every model, and the bases rank the same way everywhere, with ordering by first-order loss effect worse than ordering by reconstruction. But the size of the effect does not transfer. An independent control on an ordinary per-head KV cache agrees: a coordinate-basis overstatement of 1.65-3.22x on our three models but only 1.06-1.30x on public GPT-2 checkpoints (Appendix D).

The economical reading is that 3,000-8,000-step TinyStories models are unrepresentatively compressible in a rotated basis, and that in a production-trained MLA model the tail requirement is mostly a property of the latent code itself. This is a caution about what our small models can say on their own, not about the spectrum, which the production model reproduces. Because the deployed packed cache and the router of Section 6 both use the coordinate basis, the coordinate-basis spectrum is also the one that bounds them.

6 Can a Learned Router Exploit the Spectrum?

The spectrum says that average-case truncation is cheap and worst-case truncation is expensive, and that the requirement varies widely across tokens. A per-token allocator should be able to exploit that variation. We test whether a learned one does, and against what.

6.1 Router and training

Layer 0 runs at full width. A two-layer MLP reads the normalized state entering layer 1, which has already been updated by full-width layer-0 attention over the context, and selects one tier for the token; all downstream layers use that tier. Keeping layer 0 full avoids the circularity of compressing the state that decides the compression. The 30M and 250M-coarse policies use four tiers (6.25%, 25%, 62.5% and 100% of d_c); the 122M and 250M-fine policies use 14 and 18 tiers on the grid of Section 3.3.

Deployment uses the hard choice $y_t = \mathrm{onehot}(\arg\max a_t)$. Training passes gradients through it with a straight-through estimator [28], $\hat{y}_t = y_t + p_t - \mathrm{stopgrad}(p_t)$ with $p_t = \mathrm{softmax}(a_t/\tau)$, so that the forward pass is exactly the deployed hard rollout, and minimizes

Table 4: Pre-registered fresh-window confirmations of the router against exact-byte random placement, 24 sequences per configuration. Wins count sequences on which the router has the lower loss.

Configuration	Mean rate	Bytes vs full MLA	Δloss vs full	Router − random	95% CI	Wins	Result
30M, 4 tiers	145.78 / 256	68.80%	+0.1001	**-0.01959**	[-0.02911, -0.00938]	20/24	pass
122M, 14 tiers	206.93 / 384	61.46%	+0.1823	**-0.03251**	[-0.04575, -0.01958]	21/24	pass
250M, 4 tiers	317.15 / 512	66.82%	+0.1158	+0.00190	[-0.00222, +0.00610]	13/24	**fail**
250M, 18 tiers	278.67 / 512	60.19%	+0.1659	**-0.01165**	[-0.01911, -0.00394]	17/24	pass

$$\mathcal{J}(\theta) = \mathcal{L}_{LM}(\hat{y}) + \lambda \frac{1}{T d_c} \sum_t \sum_k p_{t,k} \mathcal{T}_k$$

with the base model frozen. Isolated-token capacity labels, the obvious supervised target, are not joint-rollout labels: an early router trained on them collapsed toward the smallest tier when every token was compressed at once, and second-order interaction measurements confirm that isolated labels are misaligned with joint truncation, although only locally (Appendix E). λ was chosen on four validation sequences as the cheapest candidate within +0.15 nat of full MLA that also beat an exact-byte static control. Appendix A gives the grids, the chosen λ values and the initialization.

6.2 Against random and shuffled placement at equal bytes

The primary control spends the router's exact per-sequence rate sum $R = qT + m$ by giving rate $q + 1$ to m random positions and q to the rest, averaged over 20 draws: the same bytes as the router (Proposition 1), placed independently of content. A configuration passes when the 95% interval of the paired per-sequence router-minus-control loss lies below zero (Table 4).

The router passes at 30M and 122M (exact one-sided sign-flip $p = 0.00066$ and 0.000049). At 250M with the same four coarse tiers as 30M it fails, with the point estimate in the wrong direction. Because 30M used coarse tiers and 122M fine ones, that failure confounds scale with tier resolution, so we froze a second 250M protocol with 18 fine tiers mirroring 122M's scheme. It passes (−0.01165 nat, $p = 0.0044$): at $d_c = 512$, four tiers are too coarse for the router to beat random placement, and the initial 250M failure is at least partly a tier-resolution artifact of our design, not a scale limit. This second protocol was designed after the coarse result was known and reused its 24 fresh windows, so it is weaker evidence than a confirmation on new windows (Appendix B). We report both protocols, and we do not combine the four into one multi-scale claim, which no protocol pre-specified.

A secondary control permutes the router's own tier multiset across positions, keeping bytes and the tier histogram. The router beats it everywhere, by -0.07989 nat (30M), -0.13907 (122M), -0.01765 (250M, 4 tiers) and -0.03551 (250M, 18 tiers), all with intervals excluding zero. Where the router puts its tiers matters even in the one configuration whose overall allocation does not beat random. Figure 2 shows the per-sequence differences and the loss-bytes operating points at 30M and 122M.

6.3 Against simple causal rules at equal bytes

Random and shuffled placement are weak baselines. A cheap causal rule, depending only on the current token or its absolute position, might place the same bytes as well. We fit four such rules as predictors of the router's own tier on the 16 training sequences: absolute position, token identity (a frequency-smoothed per-token lookup table), inverse token frequency, and coarse token type. Each rule receives only the router's per-sequence rate sum and distributes it by its own score, unrestricted to the router's tiers, so every comparison

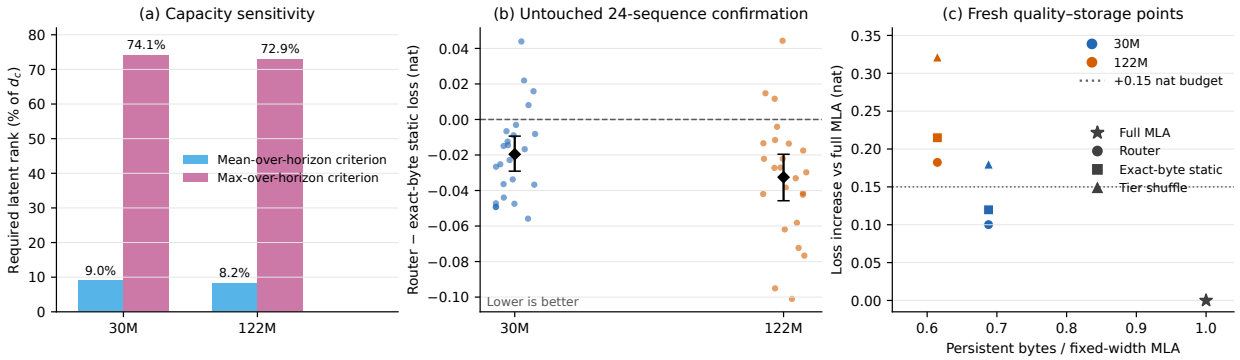


Figure 2: (a) Mean required rate under the mean and maximum criteria, averaged over 768 probes per scale. (b) Per-sequence router-minus-control loss differences at 30M and 122M; dots are sequences, diamonds means, error bars paired sequence-bootstrap 95% intervals. (c) Mean fresh-sequence loss increase over full MLA against cache bytes; both controls have exactly the router's per-sequence bytes. Points are evaluated configurations, not an interpolated curve.

Table 5: Router minus rule loss (nat) at exactly the router's bytes. Bold: the router wins (95% interval below zero); "loses": the interval is above zero; all other entries are ties.

Rule	30M, 4 tiers	122M, 14 tiers	250M, 4 tiers	250M, 18 tiers
Position	**-0.0225**	**-0.0359**	+0.0012	**-0.0133**
Token type	**-0.0146**	**-0.0284**	+0.0009	**-0.0114**
Token frequency	-0.0084	-0.0045	+0.0071 (loses)	-0.0001
Token identity	+0.0024	-0.0033	+0.0045 (loses)	+0.0022

is at exactly the router's bytes. These comparisons were run after the confirmations, on the same already-opened windows, and are exploratory.

The router beats position and token type in every configuration except 250M with four tiers, where it also fails the random-placement test. It never beats token identity, which ties it in three configurations and beats it in the fourth. Appendix C gives the intervals and a second, tier-restricted variant of this comparison.

The supported reading is narrower than "contextual routing works". At equal bytes the router places capacity better than position- and type-based rules, but no better than a per-token table fitted to its own decisions on 16 sequences. What it learned is largely expressible as token identity, and we have not shown that context beyond the current token improves placement. Given Section 5.2 this is perhaps unsurprising: the requirement that matters is set by a token's worst future offset, which a router reading only the past cannot observe.

6.4 The quality budget does not transfer reliably

The 30M and 250M-coarse policies stay within the +0.15-nat selection budget on fresh sequences (+0.1001 and +0.1158 nat); the 122M and 250M-fine policies do not (+0.1823 and +0.1659, against +0.1408 and +0.1483 on validation). Budget calibration and allocation quality are separate axes: 122M misses its budget but keeps its advantage over random placement, while 250M-coarse meets its budget and loses that advantage. A deployed system would need a larger calibration set, a risk margin, or fallback to a higher tier.

Table 6: Prefill followed by 128 decode steps on a Tesla T4, router packed cache against full-width MLA.

Model	Cache bytes, router / full	Peak allocated, router / full	Decode step, router (vs full)
30M	67.4%	99.7%	18.83 ms (2.42x)
122M	59.6%	99.3%	42.11 ms (2.37x)

7 What the Packed Cache Costs

We benchmarked prefill followed by 128 incremental decode steps on a Tesla T4, comparing full-width dense MLA, a uniform packed cache at the router's realized mean rate, and the router's packed cache.

Measured cache bytes track Proposition 1's formula. Peak allocated memory is essentially unchanged, because at these model sizes and context lengths it is dominated by weights and the prefill's vocabulary-sized logits, not by the cache. Decode is 2.4x slower per step. Most of the remaining cost is the dense reconstruction of the latent before each attention call, which is inherent to a persistent representation that is packed: removing it requires computing the per-head projections directly on the ragged buffer, a kernel we have not written. Appendix F documents how the latency was brought down from an initial 168-201x and verifies that no step changed the computation. We therefore claim byte-exact persistent-cache savings, not peak-memory or throughput gains.

8 Limitations

Scale and data. Our three models are small and trained on TinyStories, with one checkpoint, one training run and one router seed each; intervals do not include training variability, and model size is confounded with training budget. The one external replication, DeepSeek-V2-Lite, covers the spectrum and the basis ablation on one checkpoint, one text distribution and 384 probes; the router was not trained or evaluated on it.

What r^* is. Rank-loss curves are frequently nonmonotone (Appendix G), so r^* is a conservative suffix rule on a discrete grid, not a smooth or uniquely identified intrinsic rank. It measures isolated single-token truncation; joint truncation interacts locally (Appendix E).

Router evaluation. The 250M-fine confirmation reused windows whose coarse-grid outcomes had been seen, and the heuristic comparisons are exploratory. We have not compared against separately optimized global-budget policies, eviction or quantization baselines, or a router and packed cache operating in a rotated basis. Every compressed policy increases loss over full MLA, and task-level generation quality is not evaluated.

Auditability. For 30M and 122M the per-probe raw deltas behind the spectrum were computed on ephemeral cloud storage; only hash-verified aggregate summaries were recovered, so those two spectra cannot be re-audited per probe. The 250M and DeepSeek-V2-Lite raw records are kept and authenticated by hash.

9 Conclusion

Asking how much of an MLA latent each token needs has no single answer: it depends on how much of the token's future one insists on protecting. We showed that this dependence is exact and monotone (Proposition 2), and measured it. Safe on average costs 7-9% of the latent; safe at the worst of the next 32 positions costs 72-79%, with a nearly constant normalized premium across an 8x range of model size and in a production-trained model. The premium is not paid for rare tokens but for almost every token, whose small average

future effect hides harm at some offset. In small models much of that requirement can be rotated away; in DeepSeek-V2-Lite most of it cannot.

For allocation the implication is a warning. A learned per-token router with a genuinely packed cache does beat random and position- or type-based placement at exactly equal bytes, but it does no better than a per-token lookup table, and a router that sees only the past cannot observe the worst future offset that sets the requirement. Two directions follow: allocation criteria and training signals that target the tail directly, and basis choices that reduce the requirement itself, tested on production-trained MLA models where, by our measurements, the room for the latter is smaller than small models suggest.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," NeurIPS, 2017.

[2] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," arXiv:2405.04434, 2024.

[3] DeepSeek-AI, "DeepSeek-V3 Technical Report," arXiv:2412.19437, 2024.

[4] T. Ji et al., "Towards Economical Inference: Enabling DeepSeek's Multi-Head Latent Attention in Any Transformer-based LLMs," arXiv:2502.14837, 2025.

[5] Z. Yang et al., "Lossless KV Cache Compression to 2%," arXiv:2410.15252, 2024.

[6] F. Meng et al., "TransMLA: Multi-Head Latent Attention Is All You Need," arXiv:2502.07864, 2025.

[7] Z. Cai et al., "EG-MLA: Embedding-Gated Multi-head Latent Attention for Scalable and Efficient LLMs," arXiv:2509.16686, 2025.

[8] Z. Zhou et al., "CARE: Covariance-Aware and Rank-Enhanced Decomposition for Enabling Multi-Head Latent Attention," arXiv:2603.17946, 2026.

[9] D. S et al., "Through the Bottleneck: How Multi-head Latent Attention Separates Content from Position in Language Models," arXiv:2607.23054, 2026.

[10] Z. Zhang et al., "H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," NeurIPS, 2023.

[11] G. Xiao, Y. Tian, B. Chen, S. Han and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," ICLR, 2024.

[12] Y. Li et al., "SnapKV: LLM Knows What You are Looking for Before Generation," NeurIPS, 2024.

[13] Z. Cai et al., "PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling," arXiv:2406.02069, 2024.

[14] Z. Liu et al., "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," ICML, 2024.

[15] C. Hooper et al., "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," NeurIPS, 2024.

[16] C.-C. Chang et al., "Palu: Compressing KV-Cache with Low-Rank Projection," ICLR, 2025.

[17] R. Zhang et al., "LoRC: Low-Rank Compression for LLMs KV Cache with a Progressive Compression Strategy," arXiv:2410.03111, 2024.

[18] Y. Feng et al., "Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference," arXiv:2407.11550, 2024.

[19] Z. Liu et al., "Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time," NeurIPS, 2023.

[20] S. Ge et al., "Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs," ICLR, 2024.

[21] U. Saxena et al., "Eigen Attention: Attention in Low-Rank Space for KV Cache Compression," arXiv:2408.05646, 2024.

[22] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," arXiv:1911.02150, 2019.

[23] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," EMNLP, 2023.

[24] A. Liu et al., "MiniCache: KV Cache Compression in Depth Dimension for Large Language Models," NeurIPS, 2024.

[25] P. Nawrot et al., "Dynamic Memory Compression: Retrofitting LLMs for Accelerated Inference," ICML, 2024.

[26] A. Graves, "Adaptive Computation Time for Recurrent Neural Networks," arXiv:1603.08983, 2016.

[27] D. Raposo et al., "Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models," arXiv:2404.02258, 2024.

[28] Y. Bengio, N. Léonard and A. Courville, "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation," arXiv:1308.3432, 2013.

[29] R. Eldan and Y. Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?," arXiv:2305.07759, 2023.

[30] A. Kusupati et al., "Matryoshka Representation Learning," NeurIPS, 2022.

[31] S. Merity, C. Xiong, J. Bradbury and R. Socher, "Pointer Sentinel Mixture Models," ICLR, 2017.

[32] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI technical report, 2019.

[33] R. T. Rockafellar and S. Uryasev, "Optimization of Conditional Value-at-Risk," Journal of Risk, 2(3):21-42, 2000.

A Router and selection details

The 4-tier grid is $\{16, 64, 160, 256\}$ at 30M and $\{32, 128, 320, 512\}$ at 250M. The fine grid runs from 16 in steps of 16 up to 64 and in steps of 32 thereafter, giving 14 tiers at 122M ($d_c = 384$) and 18 at 250M ($d_c = 512$). Candidate λ values were trained on 16 sequences and selected on four validation sequences by the rule of Section 6.1. The selected values are $\lambda = 0.4$ (30M), 0.8 (122M), 0.4 (250M, 4 tiers) and 0.8

(250M, 18 tiers); the two 250M policies were trained and frozen independently. The 122M and 250M-fine routers start from a random initialization; the other two start from the isolated-position supervised router. At 122M the 4-tier grid produced no candidate within the validation budget, and the 14-tier grid was adopted before any fresh-window run; this is consistent with the tier-resolution explanation of Section 6.2, but means the 30M-coarse / 122M-fine contrast was not designed in advance. Four older development sequences were observed repeatedly during method development; they are exploratory and enter no confirmatory claim.

B Confirmation protocol and provenance

For 30M and 122M, commit `a4bcc7f` froze each policy file and its SHA-256 hash, the sampling seed (91,827), 24 fresh sequences, 20 control permutations, the primary endpoint and the success rule, before any fresh-window result existed (`notes/fresh_confirmation_protocol.md`). The machine-readable manifest that later scripts authenticate against (`experiments/fresh_confirmation_manifest.json`) was written afterwards, in the commit of the 30M/122M results (`080af82`); it restates the frozen values, but the oracle and summary hashes it adds were not part of the 30M/122M freeze. The 250M-coarse and 250M-fine entries were each committed to the manifest before their runs (`1ee1528`, `010e670`).

The two departures from a clean pre-registration are these. First, the 250M-fine entry was added after the 250M-coarse result was known, to test the tier-resolution explanation, and reused the same 24 windows (same seed and exclusion set). Its policy was trained and selected on the original training and validation sequences only, so the reuse cannot have tuned it, but it is a second look at windows whose coarse outcomes had been seen. Second, the original cloud commands preceded the later manifest-enforcement arguments; the frozen protocol commit, complete logs and audits bind those results, and future runs fail unless their manifest inputs match.

Fresh windows come from the held-out region and are separated from all 24 earlier oracle and development windows, and from each other, by at least `block_size + 1` tokens. `experiments/audit_fresh_confirmation.py` reconstructs the seeded sampler and recomputes every statistic from the stored per-sequence rows; it checks sampling, byte equality and arithmetic, but does not rerun the model. `experiments/audit_joint_training_replay.py` compared a separately retrained policy with the frozen one and found a bit-for-bit match; the retrained files were not committed, so repeating that check requires retraining. Exact one-sided sign-flip tests enumerate all sign assignments, which assumes exchangeable signs under the null.

C Heuristic comparison details

Each rule scores tokens; in the main comparison (Section 6.3) a token gets $r_t = \lfloor s_t + b \rfloor$ clipped to $[1, d_c]$, with b solved per sequence and the integer remainder given to the largest fractional parts, so that the rule spends exactly the router's rate sum without being restricted to the router's tiers. In the second variant the rule receives the router's realized tier multiset and only decides which positions get which tier (highest score, highest tier; ties broken at random and averaged over five draws), the frame of the shuffle control. Every evaluation asserts byte equality with the router.

With its own histogram the router wins 6 of 16, ties 8 and loses 2. Restricted to the router's tier multiset the rules lose 14 and tie 2 (30M token identity and 250M-coarse token identity), but that variant handicaps them: the router chose its histogram together with its placement, and a near-uniform histogram is itself better than a shuffled router histogram (Section 6.2). We therefore treat the own-histogram comparison as the operative one. An earlier version of this comparison let each rule run at its own validation-selected bytes, which differed from the router's by -6.5% to +13.0% and decided almost every outcome by budget rather than placement; it is superseded by the byte-matched results here (`notes/code_review_2026-10_corrections.md`).

Table 7: Router minus rule loss (nat) for every configuration and rule, with the rule choosing its own histogram (with 95% CI) or restricted to the router's tier multiset.

Configuration	Rule	Router − rule, own histogram	95% CI	Router − rule, router's histogram
30M, 4 tiers	position	-0.0225	[-0.0326, -0.0117]	-0.0804
30M, 4 tiers	token identity	+0.0024	[-0.0079, +0.0130]	-0.0049
30M, 4 tiers	token frequency	-0.0084	[-0.0162, +0.0002]	-0.0437
30M, 4 tiers	token type	-0.0146	[-0.0241, -0.0050]	-0.0689
122M, 14 tiers	position	-0.0359	[-0.0497, -0.0227]	-0.1309
122M, 14 tiers	token identity	-0.0033	[-0.0133, +0.0065]	-0.0160
122M, 14 tiers	token frequency	-0.0045	[-0.0175, +0.0091]	-0.0418
122M, 14 tiers	token type	-0.0284	[-0.0421, -0.0151]	-0.1139
250M, 4 tiers	position	+0.0012	[-0.0031, +0.0054]	-0.0210
250M, 4 tiers	token identity	+0.0045	[+0.0012, +0.0080]	-0.0019
250M, 4 tiers	token frequency	+0.0071	[+0.0028, +0.0114]	-0.0142
250M, 4 tiers	token type	+0.0009	[-0.0035, +0.0053]	-0.0217
250M, 18 tiers	position	-0.0133	[-0.0208, -0.0050]	-0.0397
250M, 18 tiers	token identity	+0.0022	[-0.0053, +0.0100]	-0.0103
250M, 18 tiers	token frequency	-0.0001	[-0.0084, +0.0090]	-0.0164
250M, 18 tiers	token type	-0.0114	[-0.0189, -0.0039]	-0.0430

D Basis ablation details

With W_{DKV}, W_{UK} and W_{UV} bias-free, substituting $U^\top W_{DKV}$, $W_{UK}U$ and $W_{UV}U$ for an orthonormal U leaves the full-width function unchanged while making a prefix mask a projection onto $\mathrm{span}(U_{:,:k})$; this is verified at full width to at most 3.1×10^{-5} logit deviation. The "grad" basis uses eigenvectors of $\mathrm{sym}(\mathbb{E}[gc^\top])$, whose quadratic form is the mean first-order loss effect of dropping a direction; its diagonal in the coordinate basis is the signed version of the saliency used by the packed cache, so the deployed ordering is the axis-aligned special case of the same criterion. The "model metric" basis uses principal directions of $A^{1/2}\mathbb{E}[cc^\top]A^{1/2}$ with $A = W_{UK}^\top W_{UK} + W_{UV}^\top W_{UV}$, the frame that minimizes reconstruction error in the metric the model applies to the latent.

In our three models the worst-offset requirement under principal directions is 23.56%, 23.25% and 21.89% (paired falls 50.5, 49.7 and 49.9 points). Under the coordinate basis at 30M, 410 of 768 probes need at least 224 of 256 channels, against 3 under rotation; at 250M, 77 of 768 need the full latent under coordinates and none under rotation. Ordering by first-order loss effect is 10-12 points worse than ordering by reconstruction at every scale: the worst-offset criterion binds near a quarter of the width, far from where a linearization taken at full width applies. The model-metric frame wins by under a point at 30M and 250M and ties principal directions at 122M, with about half of all probes tied. Under rotated bases more probes have a nonmonotone rank curve (59% to 82% at 30M, 78% to 98% at 250M), and rotated-basis mean-criterion values sit on the grid's smallest entry and are upper bounds. The coordinate arm reproduces the main-text endpoints and histograms exactly. Table 8 gives all four bases for DeepSeek-V2-Lite.

These four bases were measured in two runs that share bit-identical evaluation tokens, probe positions and baseline losses, checked before pairing; each run computed its own calibration statistics, whose float16 GPU backward pass is not bit-deterministic.

The per-head KV control truncates the key and value of each head of an ordinary multi-head cache in coordinate and rotated bases at loss budgets of 0.05-0.5 nat. The coordinate basis overstates the required width by 1.65-3.22x on our three models but by 1.06-1.30x on GPT-2 and GPT-2-medium [32] (`notes/kv_cache_basis_generalization.md`, `notes/low_rank_does_not_mean_compressible.md`).

Table 8: DeepSeek-V2-Lite: mean suffix-safe rate as a fraction of d_c in the four bases.

k (of 32)	coord	pca	grad	model metric
32 (mean)	8.22%	5.06%	5.94%	5.19%
16	22.92%	12.72%	16.58%	13.79%
8	40.14%	25.68%	31.18%	26.68%
4	56.86%	42.36%	49.51%	43.47%
2	70.69%	56.90%	64.97%	57.44%
1 (max)	78.65%	67.34%	73.47%	67.58%
TCP	0.704	0.623	0.675	0.624

E Pairwise interaction of truncations

Isolated-token labels would be joint-rollout labels if truncation effects were additive. We measured the second-order mixed difference $L_{\{t_1,t_2\}} - L_{\{t_1\}} - L_{\{t_2\}} + L_{\emptyset}$, which vanishes for any additive distortion, over 1008 position pairs per scale. It is nonzero, so isolated labels are structurally misaligned with joint routing, but the misalignment is local. As a fraction of the individual effects it is 9.09%, 12.17% and 13.17% at 30M, 122M and 250M for adjacent positions averaged over three tested rates (16.79%, 23.28% and 22.78% at rate 16 alone), decays monotonically with separation, and falls below 1.3% beyond 32 tokens. It grows with truncation severity (5.69% at rate 16 to 1.07% at rate 224 at 30M, averaged over separations). We draw no scale trend: the tested rates are fixed absolute values and so different fractions of d_c at each scale, and with 144 pairs per cell from eight sequences and no interval, the data neither establish nor exclude growth with scale (`notes/pair_interaction_prediction5.md`).

F Latency engineering

As first written, the packed decode step was 167.9x (30M) and 201.1x (122M) slower than full MLA. Three unvectorized Python loops were responsible: `pack_latents`, `unpack_latents` and `append_packed_latents` rebuilt the whole cached history token by token on every decode step, and `forward_cached_packed` built its rank mask with a nested Python loop. Rewriting all three as vectorized operations (`repeat_interleave` with one gather; a broadcast comparison for the mask) cut the regression to 3.00x and 2.96x, verified by 500 random-trial cross-checks against the original implementations with zero mismatches. The append still repacked the whole history each step because the flat buffer was ordered (batch, sequence); reordering it (sequence, batch) makes every new token sort after every stored one, so appending becomes a concatenation. Profiling then showed that the dominant cost had been per-step defensive validation, about 200x the cost of the data movement; every check was kept but moved to where its guarantee is established. This gives the 2.42x and 2.37x of Section 7, measured as a paired benchmark against the previous version inside one T4 job (dense control moved by 1% or less), and verified by bit-identical end-to-end decode logits, ranks and cache bytes. An earlier version of the benchmark reported a 3.5-4.2% peak-memory reduction; it was a harness artifact (only the router path released its prefill logits before decoding) and disap-

pears when every path does (`notes/measured_cache_memory_latency.md`). The 30M decode runs to position 320, past its 256-token training context.

G Nonmonotone rank-loss curves and reproducibility

Raw rank-loss curves are frequently nonmonotone: 83.6%, 89.3% and 92.1% of mean-over-horizon curves and 59.4%, 69.7% and 78.0% of max-over-horizon curves at 30M, 122M and 250M have at least one rise (the grids have 10, 14 and 18 points, so these frequencies are not comparable across scales). This is why r^* uses the suffix rule. An earlier run at 250M stopped the rate grid at 384 and jumped to 512, which rounded every requirement in (384, 512] up to full width and overstated the 250M tail rates by up to 4.3 points; all 250M numbers here come from the corrected rerun with the same seeds, checkpoint, windows and channel orders. The 122M-minus-30M differences in the mean and max endpoints are -0.781 points (95% CI [-1.617, +0.076]) and -1.156 points ([-3.849, +1.611]).

Code, unit tests, every result artifact and the dated lab notebook are available at https://github.com/softkleenex/elasticmla; file paths in this paper refer to that repository. Every result file records the SHA-256 of its checkpoint, data, policy and source, and every script refuses to run when an input hash does not match the artifact that produced it.